로봇 정밀 턴에 숨어 있는 삼각함수 — 고2 '대수'로 전부 설명하기

> 로봇: SPIKE Prime, 전장 155mm, 바퀴 지름 56mm, 두 바퀴 사이 거리 82mm 코드: `MRC_trig_turn.py`

---

0. 풀려는 문제

로봇에게 "90도 돌아"라고 말할 방법이 없다. 로봇이 아는 것은 **모터를 몇 도 돌릴지**뿐이다. 그래서 질문은 이렇게 바뀐다:

> **"로봇 몸이 90° 돌려면, 바퀴는 몇 도 굴러야 할까?"**

이걸 어림짐작이 아니라 **수식으로 정확히** 구하는 것이 이 프로그램의 전부다. 필요한 수학은 고2 '대수'의 삼각함수 단원에 다 들어 있다: 호도법, 호의 길이 (s = rθ), 사인·코사인의 정의, 코사인법칙, 주기성.

---

1. 호도법 — 왜 도(°) 말고 라디안인가

각도를 재는 방법은 두 가지다.

- **도(°)**: 한 바퀴 = 360°. 익숙하지만 "360"은 사람이 정한 숫자일 뿐이다.
- **라디안(rad)**: 반지름이 1인 원에서 **호의 길이가 곧 각도**. 한 바퀴 = 2π rad.

변환은 비례식 하나면 된다:

$$\theta_{rad} = \theta° \times \frac{\pi}{180}$$

라디안을 쓰는 이유는 딱 하나 — 다음 공식이 미치도록 깔끔해지기 때문이다.

2. 호의 길이 s = rθ — 이 프로그램의 심장

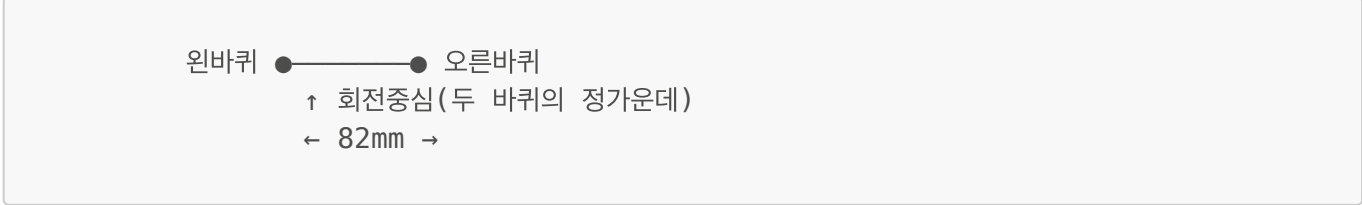
반지름 r인 원에서 중심각 θ(라디안)에 해당하는 **호의 길이**는

$$s = r\theta$$

라디안의 정의("반지름 1일 때 호 길이 = 각")를 반지름 r배로 확대한 것뿐이다. 도(°)로 쓰면 s = 2πr × (θ/360) 처럼 지저분해진다. 이래서 호도법을 먼저 배운다.

2-1. 제자리 턴(피벗)에 적용하기

로봇이 제자리에서 돌 때를 위에서 내려다보자.



- 회전중심 = 두 바퀴의 정가운데
- 왼바퀴와 오른바퀴는 각각 **반지름 41mm(= 82/2)인 원** 위를 움직인다

로봇 몸이 θ만큼 돌면, 각 바퀴는 그 원 위에서 **호의 길이만큼 굴러야** 한다:

$$s = \frac{82}{2} \times \theta_{rad}$$

계산 예: 90° 턴

- θ = 90° = π/2 rad ≈ 1.5708
- s = 41 × 1.5708 ≈ **64.4mm** ← 각 바퀴가 굴러야 할 거리

2-2. "굴러야 할 거리"를 "바퀴 회전각"으로

바퀴가 한 바퀴(360°) 돌면 나아가는 거리 = 바퀴의 원주 = π × 지름 = π × 56 ≈ 175.9mm. 비례식으로:

$$\text{바퀴 회전각} = 360° \times \frac{s}{\pi \times 56} = 360 \times \frac{64.4}{175.9} \approx 131.8°$$

2-3. 보너스: 식을 약분하면 예쁜 공식이 나온다

두 식을 합쳐서 π를 소거해 보자 (직접 해볼 것 — 수행평가 포인트):

$$\text{바퀴각} = 360 \cdot \frac{(T/2)\cdot\theta°\cdot\frac{\pi}{180}}{\pi D} = \theta° \times \frac{T}{D}$$

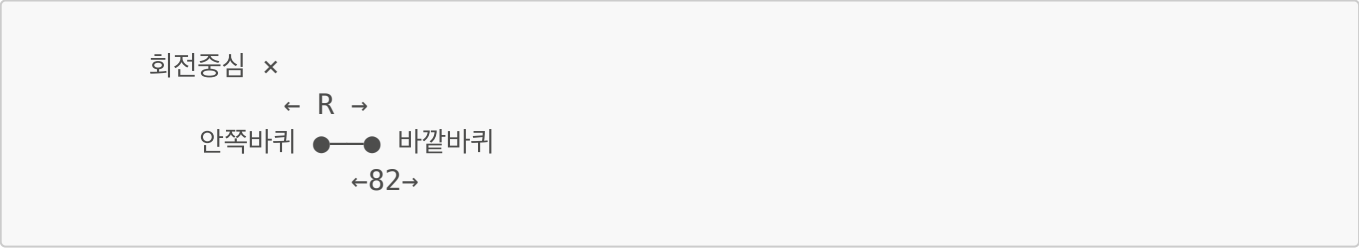
즉 **바퀴각 = 몸이 돌 각 × (바퀴사이거리 ÷ 바퀴지름)**. 우리 로봇은 T/D = 82/56 ≈ 1.464이므로, 90° 턴 = 바퀴 131.8°. π가 사라지는 게 포인트.

> 코드: `pivot_wheel_arc()`가 s = rθ, `wheel_deg_for_arc()`가 비례식 부분.

---

3. 원호 턴 — 같은 공식을 두 번 쓰면 끝

제자리가 아니라 **반지름 R짜리 곡선을 그리며** 돌고 싶다면?



회전중심에서 보면 두 바퀴의 반지름이 다르다:

- 바깥바퀴: R + 41
- 안쪽바퀴: R − 41

그래서 굴러야 할 호의 길이도 다르다 (s = rθ를 두 번):

$$s_{바깥} = (R+41)\theta_{rad}, \qquad s_{안쪽} = (R-41)\theta_{rad}$$

계산 예: R = 150mm, 90° 좌회전

- s_바깥 = 191 × π/2 ≈ 300mm → 바퀴각 614° (모터각은 기어비로 나눔)
- s_안쪽 = 109 × π/2 ≈ 171mm → 바퀴각 350°

한 가지 더: 두 바퀴가 **동시에 출발해서 동시에 끝나야** 곡선이 원호가 된다. 거리가 다른데 시간이 같아야 하므로 **속도도 호의 길이에 비례**해야 한다:

$$\frac{v_{안쪽}}{v_{바깥}} = \frac{s_{안쪽}}{s_{바깥}} = \frac{171}{300} \approx 0.57$$

> 코드: `arc_turn()`. `spd_in = speed * s_in / s_out`이 바로 이 속도비.

---

4. 사인·코사인 — "돌고 나면 어디에 있을까?" 예측

턴이 끝난 뒤 로봇의 위치를 **미리 계산**할 수 있으면, 실측과 비교해서 "수학이 맞았는지" 검증할 수 있다. 여기서 단위원 정의가 등장한다.

회전중심을 원점에 놓자. 로봇은 반지름 R인 원 위를 θ만큼 이동한다. 원 위의 점은 (R cos θ, R sin θ) — 이것이 교과서의 **사인·코사인 정의** 그대로다.

- 출발점: 각도 0 위치 → (R, 0)
- 도착점: 각도 θ 위치 → (R cos θ, R sin θ)

로봇 입장(출발할 때 보던 방향 기준)으로 바꾸면:

$$\Delta_{전방} = R\sin\theta, \qquad \Delta_{측방} = R(1-\cos\theta)$$

검산 (θ에 특수각을 넣어보기):

θ	sin θ	cos θ	전방 이동	측방 이동	직관과 비교
90°	1	0	R	R	반의 반 바퀴 → 앞으로 R, 옆으로 R ✓
180°	0	−1	0	2R	반 바퀴 → 제자리 앞이 아니라 옆으로 지름만큼 ✓

R = 150mm, 90° 턴이면 **전방 150mm, 측방 150mm** — 프로그램이 출력하는 "예측 변위"가 이 값이고, 실제 자로 재서 비교하면 된다.

> 코드: `predict_arc_move()`.

---

5. 코사인법칙 — 대회 규정(25cm) 사전 증명

로보텍스 규정: 로봇은 25cm 상자 안에 들어가야 하고 주행 중 더 커지면 안 된다. 제자리 턴을 하면 로봇의 **앞모서리가 원을 그리며 쓸고 지나간다**. 이 원이 25cm를 넘는지 미리 계산해두면 규정 걱정이 없다.

회전중심에서 모서리까지: 앞으로 a = 77.5mm(전장 절반), 옆으로 b = 61mm. 두 변이 끼인각 90°로 만나므로 **코사인법칙**으로 대각선 거리 c를 구한다:

$$c^2 = a^2 + b^2 - 2ab\cos 90°$$

cos 90° = 0이라 마지막 항이 통째로 사라지고 $c = \sqrt{a^2+b^2} \approx$ **99mm**. (피타고라스 정리가 사실 "코사인법칙의 90° 특수 경우"라는 걸 여기서 확인할 수 있다.)

스윕 지름 = 2c ≈ 197mm ≤ 250mm → **제자리 턴 합법**. 증명 끝.

> 코드: `pivot_sweep_radius()`.

---

6. 주기성 — 각도 정규화

자이로 각도는 계속 누적된다 (370°, −540° 같은 값이 나옴). 그런데 370°와 10°는 **같은 방향**이다 — 삼각함수가 주기 360°를 갖는 것과 같은 이치. 그래서 오차 계산 전에 각도를 −180°~+180° 범위로 끌어온다:

```
while deg > 180:  deg -= 360
while deg < -180: deg += 360
```

이걸 안 하면 "350° 왼쪽으로 돌아"(실제로는 10° 오른쪽이면 되는데)가 튀어나온다.

> 코드: `norm_angle()`.

---

7. 수학 + 센서 = 정밀 턴 (피드포워드와 피드백)

여기까지의 삼각함수 계산은 **완벽한 세계**의 답이다. 현실에는:

- 바퀴가 매트에서 살짝 미끄러지고
- 기어 사이에 유격(백래시)이 있고
- 바퀴 지름도 정확히 56.0mm가 아니다

그래서 프로그램은 2단으로 움직인다:

1. **피드포워드(수학)**: 삼각함수로 계산한 모터각을 그대로 명령 → 한 방에 목표의 97~98%까지 간다 (실측: 90° 명령에 92.7° — 오차 3%)
2. **피드백(센서)**: 자이로로 남은 오차를 재서, 오차에 비례하는 힘으로 미세 조정 → 마지막 1~3°를 잡는다

수학이 대부분을 해결하고 센서는 마무리만 하는 구조라서, 센서만 쓰는 방식보다 훨씬 빠르고, 수학만 쓰는 방식보다 훨씬 정확하다.

> 코드: `pivot_turn()` 1단계 = `_ff_pivot()`(수학), 2단계 = P제어 트림(센서). 시작할 때 `calibrate_turn()`이 +45° 시험 턴으로 자이로 부호까지 스스로 판별한다.

---

8. 전체 요약 — 어떤 수학이 어디에 쓰였나

고2 '대수' 개념	로봇에서의 역할	코드 함수
호도법 (라디안)	모든 각도 계산의 시작	`math.radians()`
호의 길이 s = rθ	턴에 필요한 바퀴 거리 계산	`pivot_wheel_arc()`, `arc_turn()`
사인·코사인 정의	턴 후 위치 예측 → 검증	`predict_arc_move()`
코사인법칙	회전 스윕 반경 → 규정 증명	`pivot_sweep_radius()`
주기성 (360°)	자이로 각도 정규화	`norm_angle()`

한 줄 요약: 로봇의 턴은 "원 위를 움직이는 점"의 문제이고, 원 위의 움직임을 다루는 수학이 바로 삼각함수다. s = rθ로 바퀴가 갈 거리를 구하고, sin·cos로 도착 위치를 예측하고, 코사인법칙으로 규정을 증명하고, 주기성으로 각도를 정리한다 — 교과서 한 단원이 로봇 하나를 통째로 움직인다.